



Machine Learning

NLP and LLMs

Alexander Gepperth, June 2026

What great thinkers tell us about AI

*“I think AI is one of the
biggest threats (to humans)”
(AI safety summit, UK, 2023)*



What great thinkers tell us about AI

*“I think AI is one of the
biggest threats (to humans)”
(AI safety summit, UK, 2023)*





But: let us rather ask the
experts!



Will artificial intelligences take over the world any time soon?



But: let us rather ask the experts!



Will artificial intelligences take over the world any time soon?

The idea of artificial intelligence taking over the world is a popular science fiction trope. However, in reality, it is highly unlikely that AI will take over the world anytime soon. According to a Forbes article, AI is a powerful business tool that is supporting companies and their customer service strategies ¹. Another article from Tech Business News states that AI will not take over the world and that the notion is science fiction ². While AI has been getting more powerful, it is still very narrow and specific ³. Therefore, there is no need to worry about AI taking over the world anytime soon.



But: let us rather ask the experts!



Will artificial intelligences take over the world any time soon?

The idea of artificial intelligence taking over the world is a popular science fiction trope. However, in reality, it is highly unlikely that AI will take over the world anytime soon. According to a Forbes article, AI is a powerful business tool that is supporting companies and their customer service strategies ¹. Another article from Tech Business News states that AI will not take over the world and that the notion is science fiction ². While AI has been getting more powerful, it is still very narrow and specific ³. Therefore, there is no need to worry about AI taking over the world anytime soon.



ChatGPT/... examples



The code you provided is a simple implementation of a linear classifier using gradient descent. It takes a dataset of handwritten digits from the MNIST dataset and trains the classifier to classify new digits. The code defines a `LinClass` class that encapsulates the model parameters and training process. The `train_step` method performs a single gradient descent step, and the `train` method runs the training process for a specified number of iterations.

Here's a breakdown of the code:

1. Importing Libraries:

The code imports the `numpy` library for numerical operations, `requests` for making HTTP requests to download the MNIST dataset, and `os` for interacting with the filesystem.

2. Class Definition:

The `LinClass` class defines the linear classifier model. It has the following attributes:

- `W`: A 2D NumPy array of weights with dimensions (input_dim, output_dim).
- `b`: A 1D NumPy array of biases with dimensions (1, output_dim).
- `X`: The training data matrix with dimensions (N, input_dim).
- `T`: The training labels vector with dimensions (N, output_dim).
- `N`: The number of training examples.

3. Loss Function:

The `loss` method calculates the cross-entropy loss between the model's predictions and the ground-truth labels. It takes the predictions (`Y`) and labels (`T`) as input.

4. Gradient Calculation:

The `ddW` and `ddW` methods calculate the gradients of the loss function with respect to the weights (`W`) and biases (`b`), respectively. They take the predictions (`Y`), labels (`T`), and input data (`X`) as input.

5. Softmax Function:

The `S` method applies the softmax function to a vector of inputs (`X`). It normalizes the input vector to a probability distribution.

6. Forward Pass:

The `f` method performs the forward pass, which calculates the model's predictions for a given input vector (`X`). It applies the softmax function to the dot product of the input vector and weights (`W`) plus biases (`b`).

7. Training Step:

The `train_step` method performs a single gradient descent step to update the weights and biases of the model. It takes the input data (`X`), labels (`T`), and learning rate (`eps`) as input. It calculates the loss, gradients, and updates the weights and biases accordingly.

8. Training Loop:

The `train` method runs the training process for a specified number of iterations (`max_it`). It calls the `train_step` method for each iteration.

9. MNIST Data Loading:

The code checks for the presence of the MNIST dataset (`mnist.npz`) in the current directory. If it's not present, it downloads the dataset from a specified URL and saves it locally.

10. Model Initialization and Testing:

The code initializes a `LinClass` instance using 1000 training examples from the MNIST dataset (`train[0:N]` and `train[0:N]`). It creates a test input and output vector (`x` and `y`), calculates the softmax predictions and verifies that they sum to 1. It also calculates the cross-entropy loss for a test input and label pair. Finally, it trains the model for 100 iterations with a learning rate of 0.01.

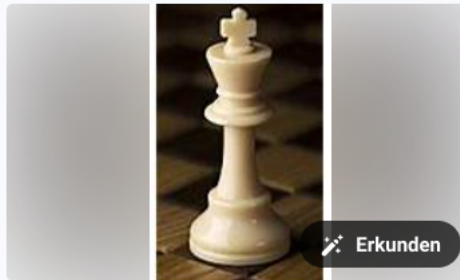
Stupid answers by LLMs



Wenn in einem Schachspiel mein weißer König auf a1 steht, und je ein schwarzer Turm auf e1 und d2: wie kann ich den Turm auf d2 schlagen?

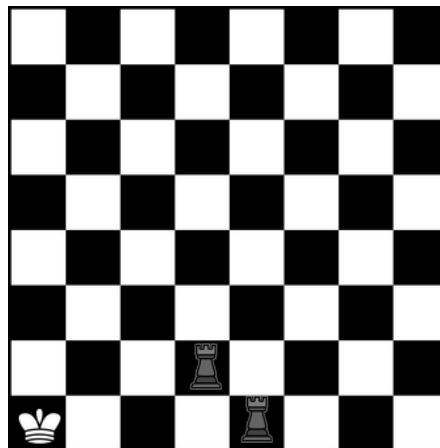
Wenn dein weißer König auf a1 steht und ein schwarzer Turm auf d2, kannst du den Turm auf d2 schlagen, indem du deinen König auf d2 ziehst. Beachte jedoch, dass dies nicht empfohlen wird, da dein König dadurch in eine gefährliche Position gerät. Es ist ratsam, deinen König an einen sichereren Ort zu bewegen, bevor du den Turm schlägst.

Ich hoffe, das hilft dir weiter. Wenn du weitere Fragen hast, lass es mich gerne wissen!

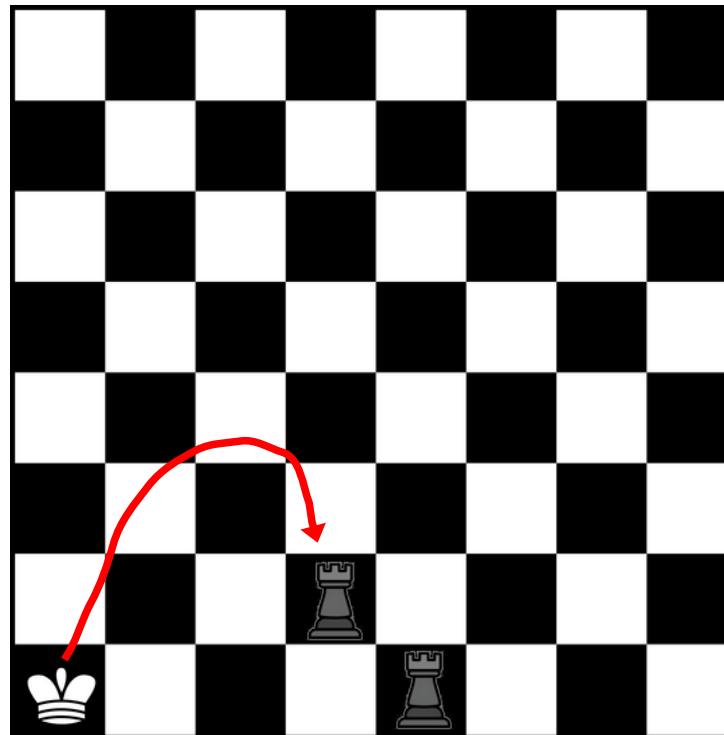


Weitere Informationen: [1. schach-tipps.de](#) [2. chess.com](#) [3. schachmatt.net](#) [+4 mehr](#)

2 von 10 ●



Stupid answers by LLMs







Basic LLM concepts

- Basic principle: text statistics by tables
- Tokenization
- Word embeddings
- Language Transformers
- Visual transformers



Text statistics

- Known since the 70s: text properties can be encoded by statistics
- Model scenario for LLM approach: tables!
 - cond. probabilities of char. c conditioned on preceding characters c_1, c_2, c_3 : $p(c|c_1, c_2, c_3)$
 - approach: build table S of shape $[K, K, K, K]$ (K is vocabulary size: a-z, space, period, ...)
 - table S contains **counts** → cf class on probabilities



Text statistics by tables

- Requires: tokenization $\rightarrow$ map characters to table indices $1, \dots, K$
- Entries of S: S_{abcd} represents # of events where sequence of character indices a, b, c, d was observed **in a given training corpus**
- Fill S using statistics in a given text corpus: learning!



Text statistics by tables

- Elementary probability theory:
 - joint probability of character indices $abcd$ occurring in sequence: $S_{abcd}/N = p_S(a, b, c, d)$
 - conditional probability:

$$p_S(d|a, b, c) =$$



Text statistics by tables

- Elementary probability theory:
 - joint probability of character indices $abcd$ occurring in sequence: $S_{abcd}/N = p_S(a, b, c, d)$
 - conditional probability:
$$p_S(d|a, b, c) = \frac{p_S(a, b, c, d)}{p_S(a, b, c)} = \frac{p_S(a, b, c, d)}{\sum_j p_S(a, b, c, j)}$$



Text statistics by tables

- Elementary probability theory:
 - joint probability of character indices $abcd$ occurring in sequence: $S_{abcd}/N = p_S(a, b, c, d)$
 - conditional probability:

$$p_S(d|a, b, c) = \frac{p_S(a, b, c, d)}{p_S(a, b, c)} = \frac{p_S(a, b, c, d)}{\sum_j p_S(a, b, c, j)}$$
$$= \frac{S_{abcd}}{\sum_j S_{abcj}}$$



Text statistics by tables

- How to assess the probability of an entire text $\vec{t}$, composed of tokens t_i , using the trained table?
- Assume that probabilities for individual characters are independent:

$$p(\vec{t}) = \prod_{i=4}^C p(t_i | t_{i-1} t_{i-2} t_{i-3})$$



Text statistics by tables

- How to assess the probability of an entire text $\vec{t}$, composed of tokens t_i , using the trained table?
- Assume that probabilities for individual characters are independent:

$$p(\vec{t}) = \prod_{i=4}^C p(t_i | t_{i-1} t_{i-2} t_{i-3})$$

- To prevent underflows:

$$\log p(\vec{t}) = \sum_{i=4}^C \log p(t_i | t_{i-1} t_{i-2} t_{i-3})$$



Text statistics by tables: text generation

- Start with text $\vec{t}$ of $L \geq 3$ tokens
 - compute $\vec{p}$ with $p_j = p(t_{L+1} = j | t_L, t_{L-1}, t_{L-2})$ for all possible tokens j
 - draw a random token according to the discrete probability distribution, append to $\vec{t}$
 - repeat



Assignment 1

- Use the provided “Shakespeare” corpus to train a table S
- Assess the probability and log-probability of, e.g.: “to be or not to be” –vs– “xfsdgfsf”
- Generate text starting with “tomorrow” until a “.” sign is produced



Assignment 1: details

- There are 29 tokens: a-z, space, period("."), '
- To sample a single value out of K possible values from a discrete prob. distribution represented by vector p :
`np.random.choice`



Assignment 1: discussion

- This simple model shows already:
 - the nature of LLMs: is this intelligence?
 - hallucinations and randomness
 - why log-probabilities are a good choice
 - the need for tokenization (if we had longer tokens, the statistics would be more powerful)
 - the way forward: as in RL, replace tables by DNNs!



Does “understanding” arise from statistics?



Chinese room and the nature of understanding

- Do LLMs or tables “understand” text?
- What does “understanding” mean?

The “chinese room” argument

- John Searle (~1960):





The “chinese room” argument

- John Searle (~1960):
 - a person places written chinese queries in an input slot
 - on the other side, another person looks up the correct answers in a book and ...
 - ... places the answers in an output slot





The “chinese room” argument

- John Searle (~1960):
 - a person places written chinese queries in an input slot
 - on the other side, another person looks up the correct answers in a book and ...
 - ... places the answers in an output slot
- Argument: to the person outside, the answers appear intelligent, yet the person inside does not even understand chinese!





The “chinese room” argument

- John Searle (~1960):
 - a person places written chinese queries in an input slot
 - on the other side, another person looks up the correct answers in a book and ...
 - ... places the answers in an output slot
- Argument: to the person outside, the answers appear intelligent, yet the person inside does not even understand chinese → **no understanding at all!** Seite 28





The “chinese room” argument

- John Searle (~1960):
 - purely formal manipulation of symbols can never achieve true understanding
 - the appearance of intelligence is not intelligence





The “chinese room” argument

- What about LLMs?
 - in any case: how big would a lookup table have to be??
 - LLMs do not just look up answers, they can compensate for linguistic variations
 - Do they “understand” what is being said?





Assignment/suggestion for presentation

- Collect philosophical and scientific arguments arguing for and against intelligence in LLMs
- Evaluate these arguments based on your own understanding!



Tokenization



Tokenization

- Goal: text → fixed-length number sequence
- To obtain fixed length, pad text if needed!

Today, I feel happy.



START	TODAY	COMMA	I	FEEL	HAPPY	STOP	PAD	PAD
1	123	151	5	5321	11	0	3	3

- Tokens can be letters, sub-words, words, punctuation, whitespace, emojis...



Tokenization

- Map (sub-)words to integers via a **vocabulary** of a given size S
- Vocabularies are trained on data, e.g., via the byte pair encoding (BPE) algorithm
- Why training: efficiency!
- Typical dictionary sizes: ~50000



BPE (Byte Pair Encoding) tokenizers

- Algorithm for training a vocabulary:
start with a vocabulary V of basic letters,
target vocabulary size S , and data D
- Repeat until $|V|=S$:
 - find most frequent pair of consec. tokens in D
 - merge this pair, add to V
- **demonstration**



Assignment 2/suggestion for presentation

- Train a BPE tokenizer on the Shakespeare corpus. Vary the vocabulary size between 29 and 1000 and observe the learned tokens
- Use the `tokenizers` package from the HuggingFace repository
- “Train” a table on the tokens instead of characters and observe what the generated text looks like!



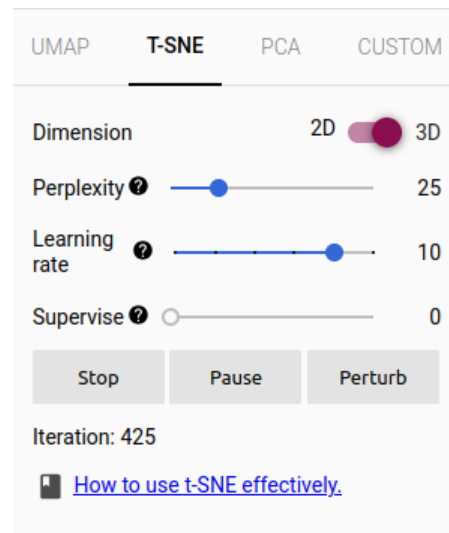
Word embeddings

- Why do we not feed a sequence of tokens into an LLM? → Token indices have no intrinsic meaning!
- Our Shakespeare 100 token vocabulary:
 - *Us*: 71
 - *We*: 72
 - *You*: 47
 - *Thou*: 94



Reminder

- Go to <https://projector.tensorflow.org/> now!





Word embeddings

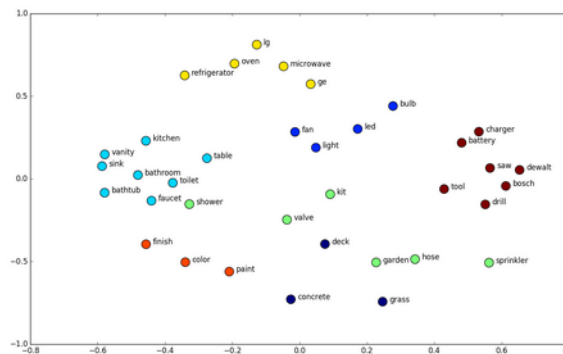
- We require an encoding of tokens t into vectors (embeddings) $\vec{v}(t)$
- Similar tokens $\rightarrow$ similar embeddings:
 $\vec{v}^T(t)\vec{v}(t')$ is high if $t \sim t'$ (else: low)
- Tokens/words can be similar in many ways:

cat~feline cat~lazy cat~cute cat~Garfield
cat~mammal cat~animal cat~quadruped
cat~meeting cat~pet cat~rocket



Word embeddings

- Requirement: expression of similarity in many independent directions
- Translates tokens t into high-dimensional embeddings $\vec{v}(t) \in \mathbb{R}^d$, $d \sim 1000$





Assignment: why embeddings should be high-dimensional

- Consider a unit vector $\vec{e}_1 = (1, 0, \dots) \in \mathbb{R}^d$ and a second, random vector $\vec{x}$
- Prove by Python implementation that the scalar product $\langle \vec{e}_1, \vec{x} + r\vec{e}_i \rangle$ is the same as $\langle \vec{e}_1, \vec{x} \rangle$ for $i > 1$, $r \sim \mathcal{U}(0, 1)$
- Prove this on paper!



Discussion

- The unit vector $\vec{e}_1$ was used for convenience, identity holds for any vector:
For any pair of vectors $\vec{y}, \vec{x} \in \mathbb{R}^d$, one can find $d-1$ vectors $\vec{x} + r\vec{y}_i^\perp$ such that the scalar product remains $\langle \vec{y}, \vec{x} \rangle$
- With increasing d , we have more freedom to vary $\vec{x}$ while not destroying similarity to $\vec{y}$



Discussion: example

- Consider the tokens **cat** and **dog**
- These two have a certain similarity c
- We would like to vary the word **dog** while keeping the similarity to **cat** at c :
 - Spaniel, Rottweiler, Dachshund, ...
 - Mammal, quadruped, vertebrate
 - ...



Training of word embeddings

- Basic idea: word similarity defined by co-occurrence in data!
- Word embeddings are trained on data:
 - Continuous Bag-of-words (CBOW) technique
 - Skip-Gram technique



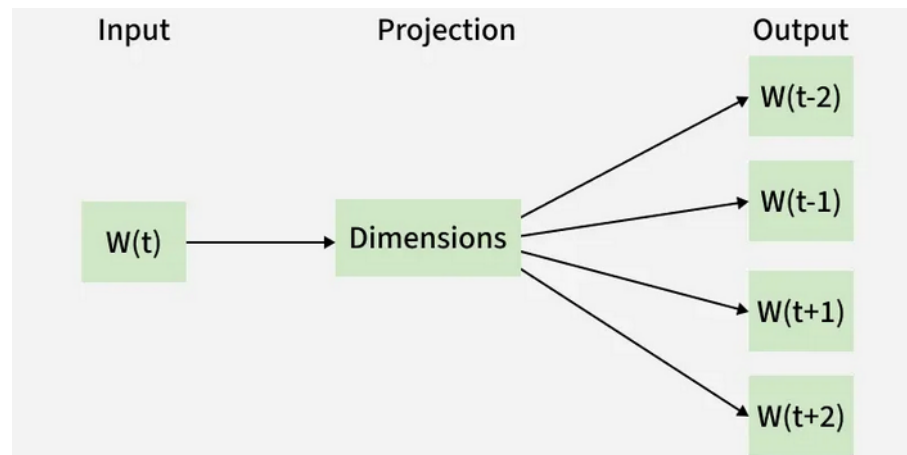
Training of word embeddings

- Training data: database of S sentences
- Each sentence s consists of T_s tokens $\rightarrow$ no fixed shape
 - pad sentences to context length $C \rightarrow (S, C)$
 - or: apply transformations during processing



Training of word embeddings: skip-gram

- For a sequence (phrase) of length C : use one **target token** in a window of size $2W+1$ to predict presence of **context tokens**





Training of word embeddings: skip-gram

- Skip-grams are pairs of target and context tokens t_T, t_C
- Embeddings $\vec{v}(t)$ such that high probabilities are obtained for all skip-grams

$$p(t_C | t_T) = \frac{\exp(\vec{v}^T(t_C) \vec{v}(t_T))}{\sum_i \exp(\vec{v}^T(t_i) \vec{v}(t_T))}$$

Window Size	Text	Skip-grams
2	[The wide road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road shimmered in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot sun].	sun, the sun, hot
3	[The wide road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	[The wide road shimmered in the hot] sun.	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the shimmered, hot
	The wide road shimmered [in the hot sun].	sun, in sun, the sun, hot



Training of word embeddings: skip-gram

- Naive Skip-Gram loss:

$$\mathcal{L}^{\text{SG}} = \sum_{t=1}^C \sum_{|j| < W, j \neq 0} \log p(t_t + j | t_t)$$

- Problem: inefficient, softmax contains sum over all tokens

$$p(t_C | t_T) = \frac{\exp(\vec{v}^T(t_C) \vec{v}(t_T))}{\sum_i \exp(\vec{v}^T(t_i) \vec{v}(t_T))}$$



Training of word embeddings: skip-gram

- Replace Skip-Gram loss ...

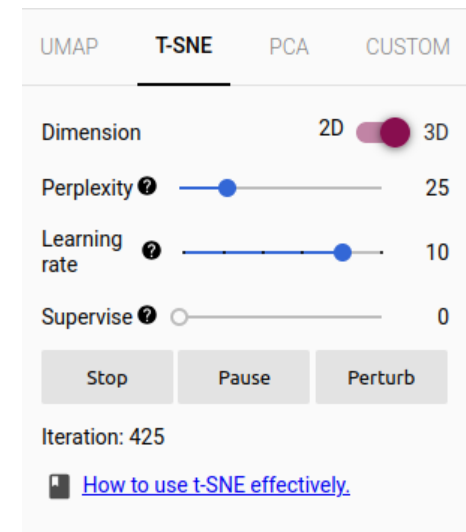
$$\mathcal{L}^{\text{SG}} = \sum_{t=1}^C \sum_{|j| < W, j \neq 0} \log p(t_t + j | t_t)$$

- .. by NCE loss:
$$\mathcal{L}^{\text{NCE}} = \sum_{t=1}^C \sum_{|j| < W, j \neq 0} \frac{p}{p + q}$$



Assignment

- Go to <https://projector.tensorflow.org/> and load the Word2Vec 10k dataset. Choose t-SNE as visualization tools
- Examine the generated clusters!
- What do you observe?





Caveats

- The field of LLMs is evolving rapidly
- Everything said here is strictly valid for GPT-2, and core principles persist, but:
 - word embeddings have evolved
 - positional encoding now done by RoPE
 - attention mechanisms have been optimized
 - ...

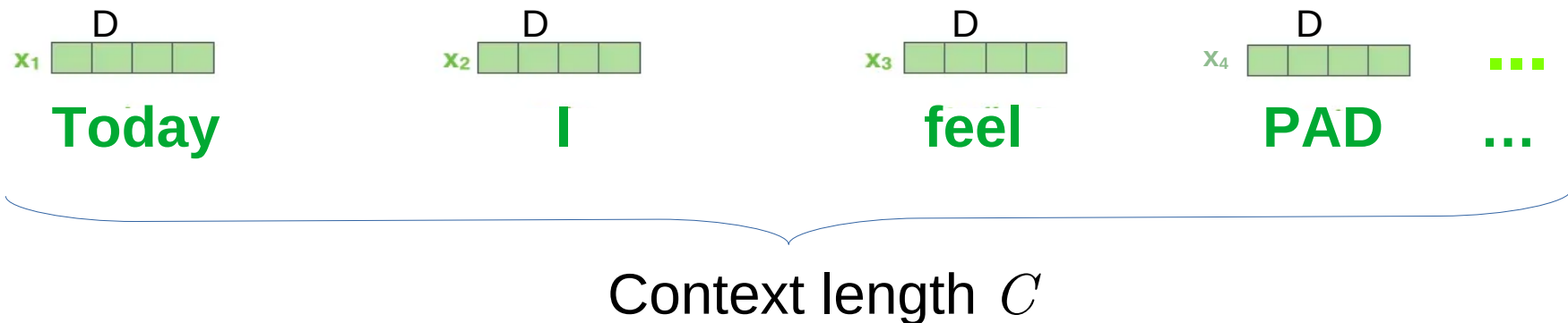


Text representation for LLMs



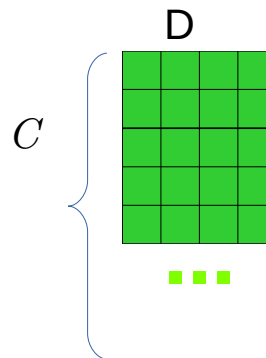
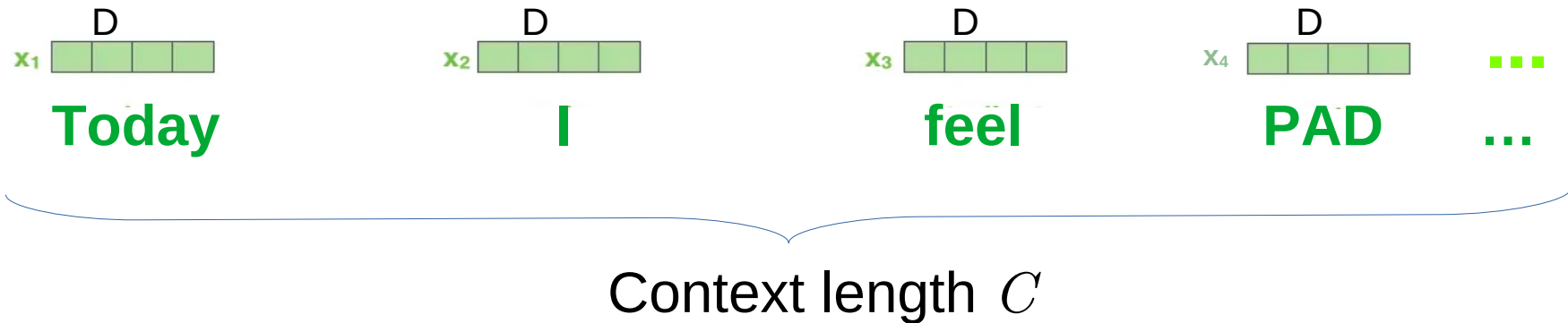
Text representation for LLMs

- Input to LLM: padded sequence of C embeddings (dimension D)
- A single sample has shape (C, D)





Text representation for LLMs

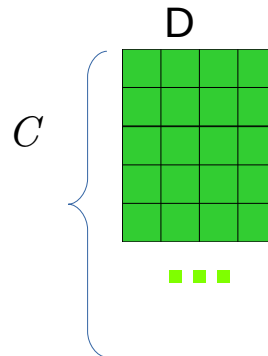




Text representation for LLMs



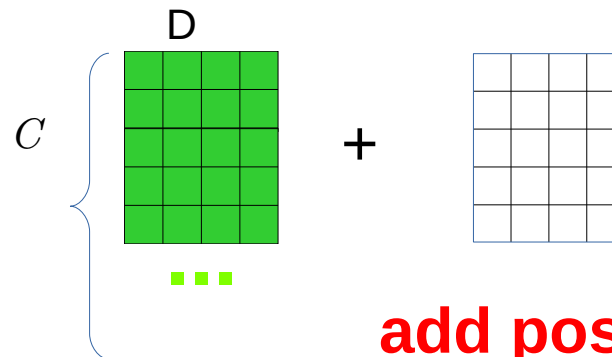
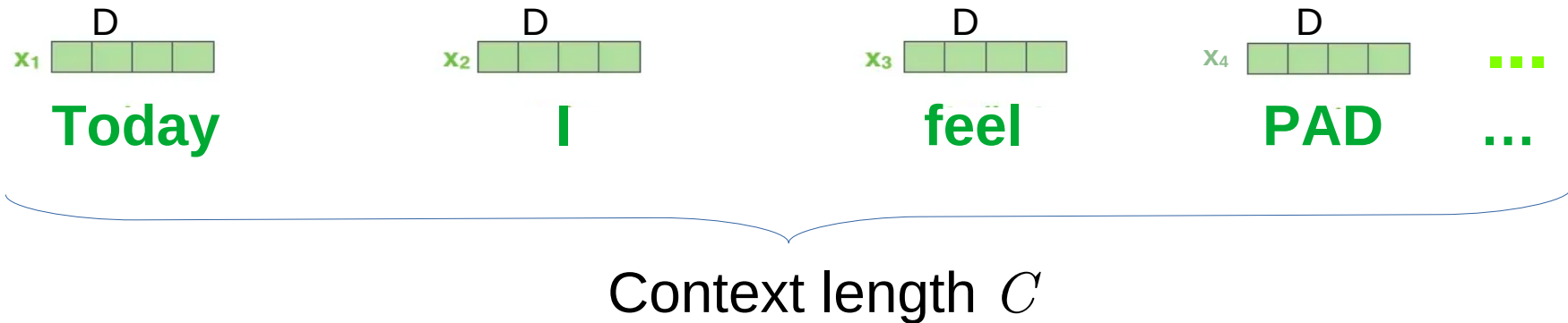
Context length C



This is one sample!



Text representation for LLMs



**add positional
encoding!**



Transformers



Transformer models

next token

- Essentially, transformers compute the conditional probability $P(u_i | u_1, \dots, u_{i-1})$
- Predict next token: L_1 loss for one sequence

$$L_1(P, \{u_1, \dots, u_C\}) = \sum_i^n \log P(u_i | u_1, \dots, u_{i-1}; \theta)$$

Diagram illustrating the components of the equation:

- Token sequence**: Points to the set $\{u_1, \dots, u_C\}$.
- Context length**: Points to the index C in the token sequence.
- trained DNN output**: Points to the index i in the summation.
- next token**: Points to the token u_i in the probability function.
- preceding tokens, padded to context length**: Points to the sequence $u_1, \dots, u_{i-1}$ in the probability function.
- Parameters**: Points to the parameter θ in the probability function.



The GPT model

- GPT: generative pre-trained transformer
- Guess the next word!
- Generated phrase is fed into decoder stack again!

$P(u_i | u_1, \dots, u_{i-1}; \theta)$
5% 2% 1% 3%
good happy tired weird ...
↑ token

Affine + Softmax

Decoder

Decoder

Decoder

Decoder

Decoder

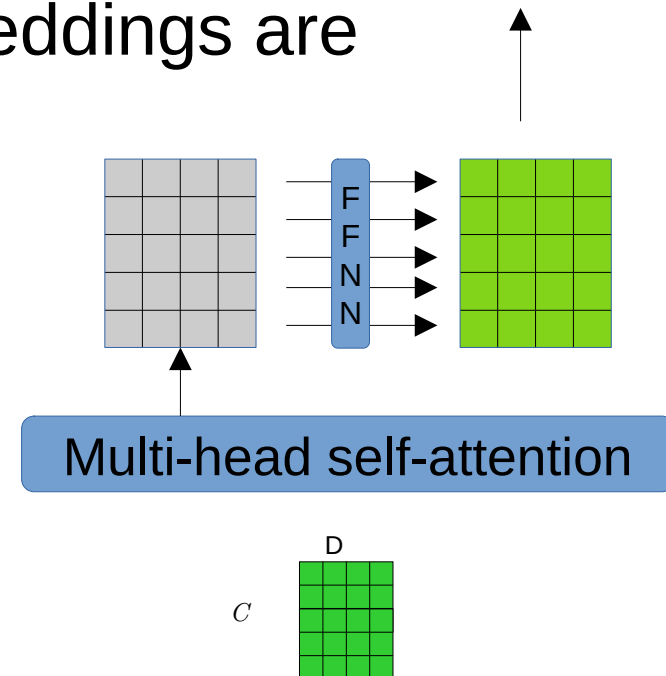
Today I feel ...

tokenization,
embedding



Decoder layers

- **Sequence-to-sequence:** Sequence of embeddings transformed into sequence of embeddings
- **Multi-head self-attention:** embeddings are processed together
- **Feed-forward stage:** embeddings are processed individually by same DNN





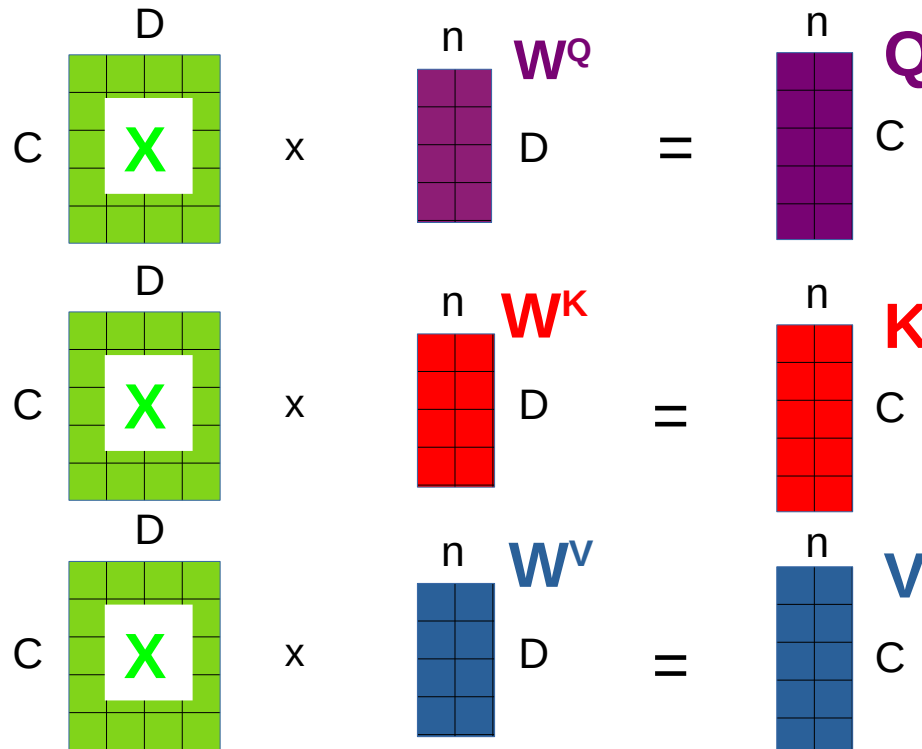
Multi-head self-attention

- Transforms a sequence of length C into another sequence of length C
- Input elements are word embeddings
- Output elements word embeddings that encode more abstract concepts
 - phrases are **transformed**



A single self-attention head

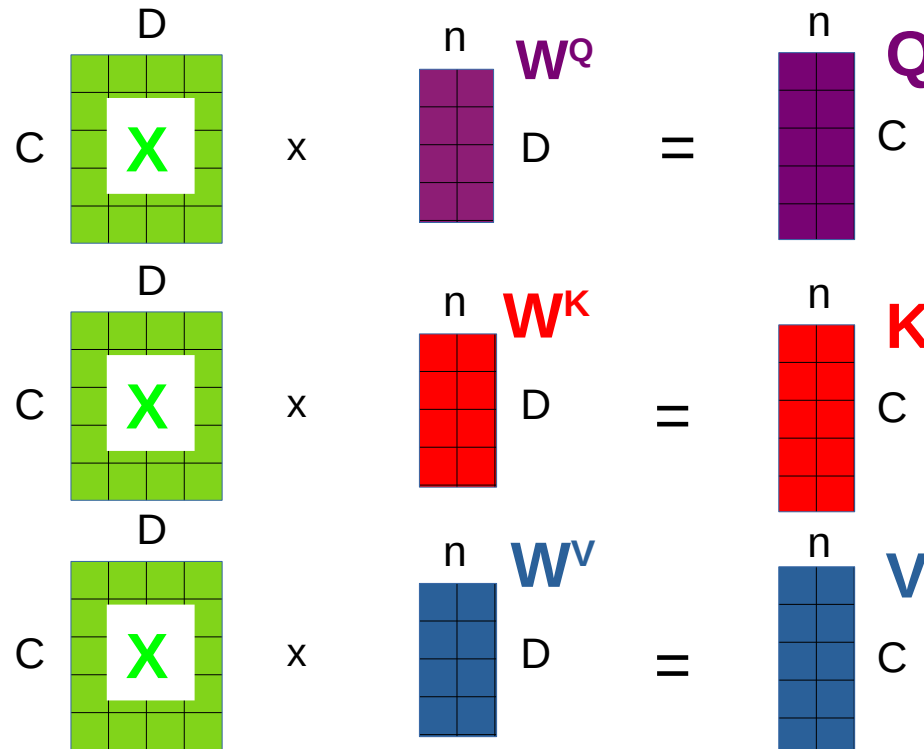
Generate (**Q**)uery, (**K**)ey and (**V**)alue sequences by trainable matrices W^Q , W^K , W^V





A single self-attention head

Generate (**Q**)uery, (**K**)ey and (**V**)alue sequences by trainable matrices W^Q , W^K , W^V



$n=64$ queries/
keys/values
applied to
each embed-
ding



A single self-attention head

- For each sequence element, a weighted sum of value vectors is computed

$$\text{softmax}_i \left(\frac{\begin{matrix} n & Q \\ & c \end{matrix} \begin{matrix} c & K^T \\ & n \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} n \\ & V \\ & c \end{matrix}$$



A single self-attention head

- For each sequence element, a weighted sum of value vectors is computed

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

i

n *c* *c* *n* *n* *c*

Q *K^T* *V*

$C \times C$ attention matrix A :
score between embedding # i
and embedding
j given by A_{ij}



A single self-attention head

- For each sequence element, a weighted sum of value vectors is computed

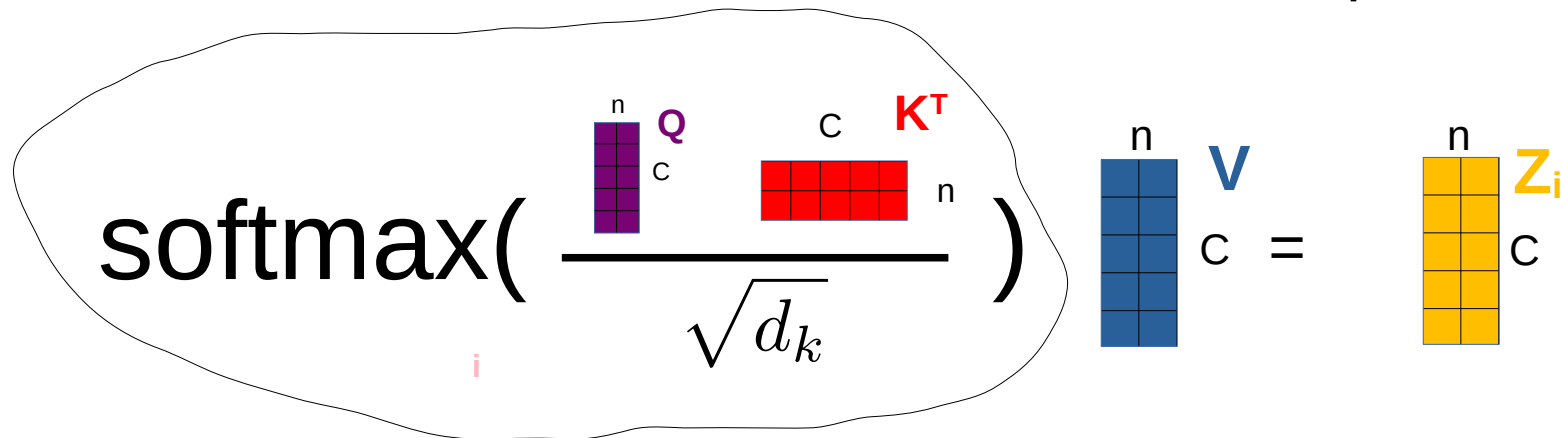
$$\text{softmax}\left(\frac{\begin{matrix} n & & c \\ \text{Q} & & \text{K}^T \\ c & & n \end{matrix}}{\sqrt{d_k}}\right) \begin{matrix} n \\ \text{V} \\ c \end{matrix} = \begin{matrix} n \\ \text{Z}_i \\ c \end{matrix}$$

$C \times C$ attention matrix A :
score between embedding # i
and embedding
j given by A_{ij}



A single self-attention head

- For each sequence element, a weighted sum of value vectors is computed



$$A_{ij} = 0 \text{ for } i < j$$

$C \times C$ attention matrix A :
score between embedding # i
and embedding
j given by A_{ij}



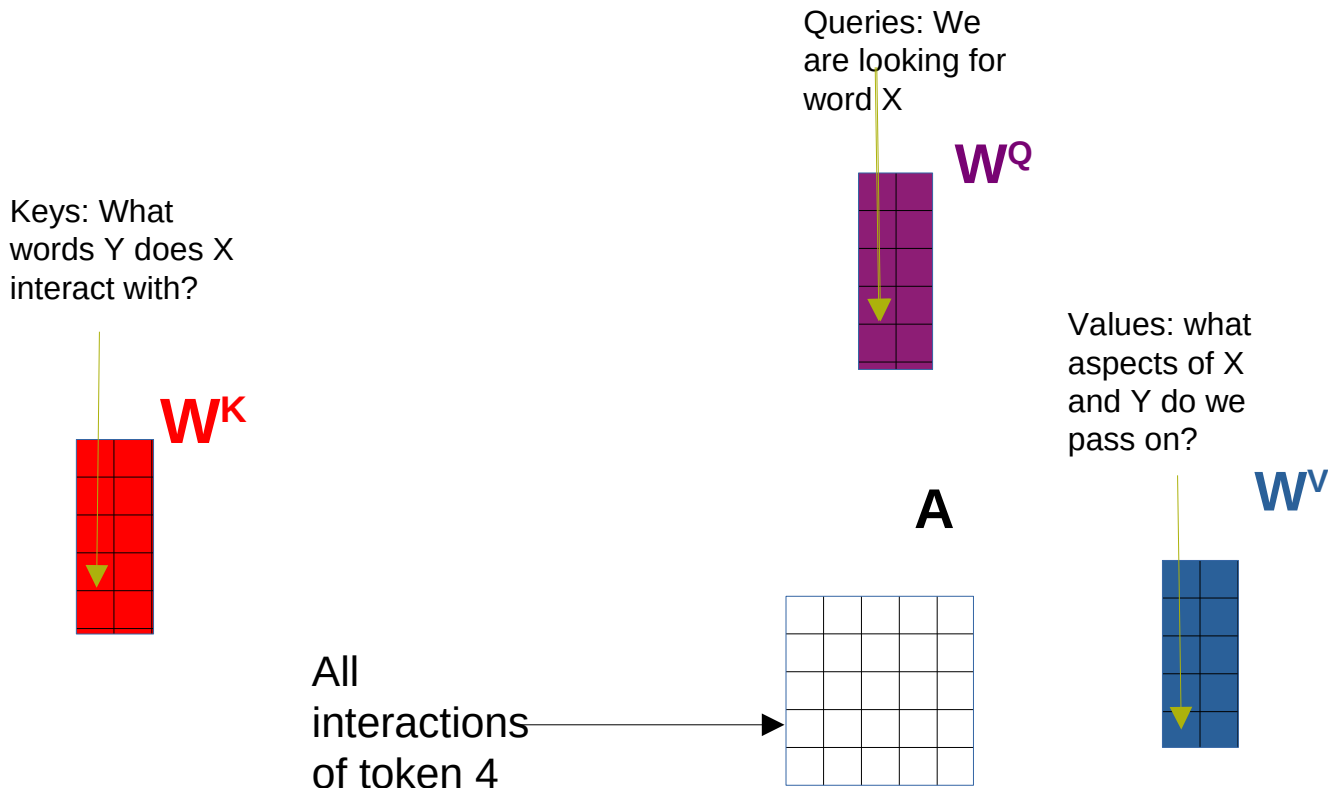
The logic behind self-attention

- Most words are not interpretable in isolation, only combining them unlock full meaning
 - “Tom is **looking at** the car”
 - “Tom is **looking for** the car”
 - “**Tom** did not know what **he** was looking for”
- This is a hierarchical process!



The logic behind self-attention

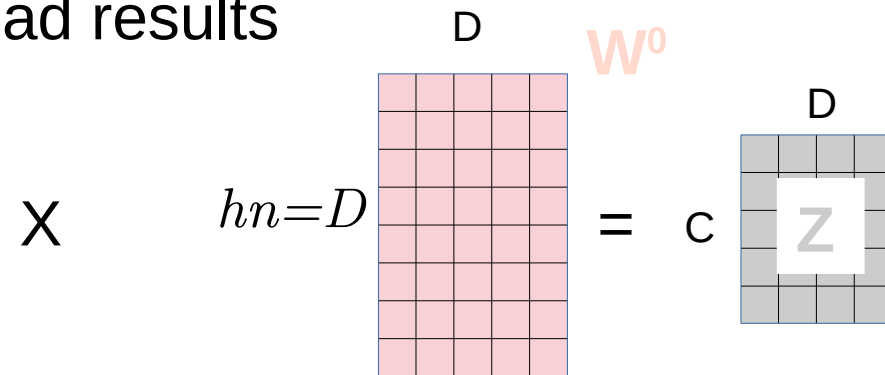
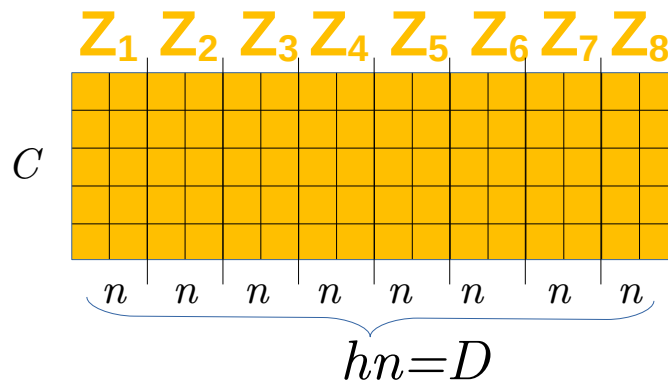
- More examples:
 - “The prof praised **Tom** because **he** was **very punctual**”
 - “Yesterday, **Apple** presented **their** new phone in California”
 - “I have a savings **account** with my **bank**”





Multi-head self-attention layers

1) Concatenate all h attention head results



2) Optionally: multiply by a weight matrix W^0 to back-project the Zs to D columns (unnecessary for simple version of GPT-2 where $hn=D$)



Feed-forward processing

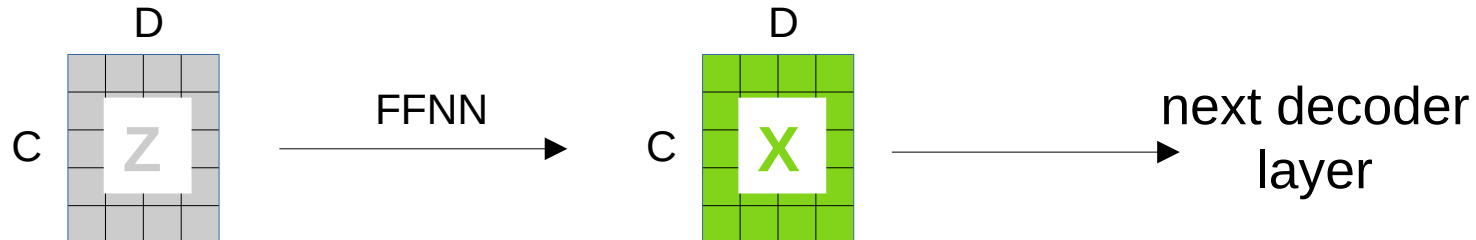
- Apply two-layer DNN to Z (result of self-attention)
- $F(Z) = r(ZW_1 + \vec{b}_1^T)W_2 + \vec{b}_2^T$
- Essentially: transform each embedding separately
- Output dimension same as input dimension





Feed-forward processing

- Apply two-layer DNN to Z (result of self-attention)
- $F(Z) = r(ZW_1 + \vec{b}_1^T)W_2 + \vec{b}_2^T$
- Essentially: transform each embedding separately
- Output dimension same as input dimension





Assignment 1

- Play around with GPT-2 at <https://poloclub.github.io/transformer-explainer/>



Assignment 2

- Download visgpt2.py from the E-learning platform
- Generate some word embeddings and measure their similarity
- Generate some sentence embeddings (word embeddings for every token in a sentence) and print them!
- Create a fake query matrix W^Q with a word from the sentence as single column and apply it to the sentence embedding. Visualize Q!
- Visualize attention matrix for all attention heads in first decoder layer for different sentence embeddings! Test which words are grouped:
 - Nouns + verbs
 - Nouns + adjectives
 - Nouns + pronouns
 - ...